

รายงาน Clean Code

(การปรับปรุงโค้ดจาก AI Generated ให้เป็นไปตามหลัก
Clean Code โดยใช้ Bloc/Cubit Pattern)

รายชื่อสมาชิกภายในกลุ่ม

- ชวาทิก ธรฤทธิ์ 6610110066
- ภัทรพล กิจเจริญ 6610110230
- ศรัณย์กร ชัยสุนทรานนท์ 6610110289

คำอธิบายเพิ่มเติม

โค้ดชุดนี้เดิมที่สร้างขึ้นด้วย AI (AI Generated) เพื่อพัฒนาแอปพลิเคชันขายของหน้าร้าน ซึ่งเมื่อทีมเรานำมาตรวจสอบ พบว่าโครงสร้างส่วนใหญ่ยังไม่สอดคล้องกับหลัก Clean Code เท่าที่ควร เช่น มีการปนกันระหว่าง Business Logic กับ UI Layer และการจัดการ State ที่กระจัดกระจาย ทีมจึงตัดสินใจ Refactor โค้ดโดยนำรูปแบบ Bloc/Cubit เข้ามาใช้เป็นแกนหลักในการจัดการ State ทั้งหมดของแอป เพื่อให้แต่ละส่วนของโค้ดมีหน้าที่ชัดเจน แยกส่วนกันทำงาน และทดสอบได้ง่ายขึ้น รายงานฉบับนี้สรุปองค์ประกอบหลัก 6 ส่วนที่ทีมได้ปรับปรุง พร้อมอธิบายว่าแต่ละส่วนทำงานอย่างไร และสะท้อนหลัก Clean Code ข้อไหนบ้าง

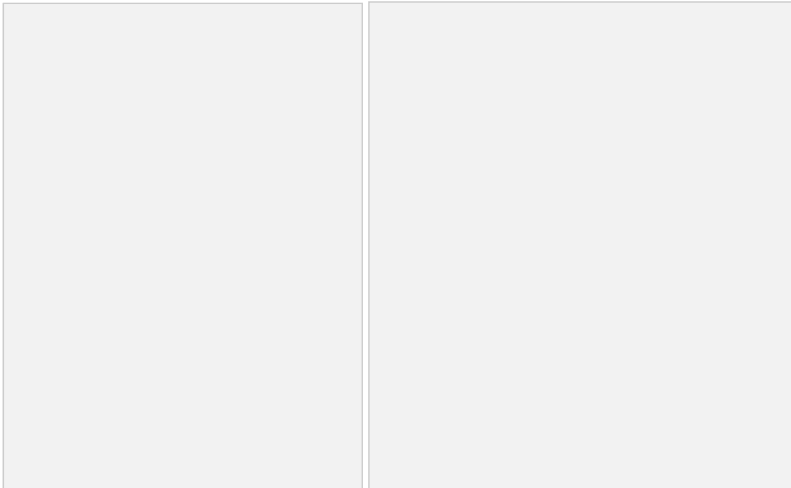
1. CartCubit

(lib/presentation/blocs/cart_cubit.dart)

ตัวนี้เป็นหัวใจของหน้าตะกร้าสินค้า ทำหน้าที่ดูแล State ทั้งหมดที่เกี่ยวข้องกับ Cart ตั้งแต่เก็บรายการสินค้า คำนวณราคารวม (Subtotal) ไปจนถึงตรวจสอบเงื่อนไขก่อนขาย เช่น สต็อกเหลือพอหรือไม่ หรือราคาที่ต่ำกว่าทุนหรือเปล่า

โครงสร้างสืบทอดมาจาก Cubit<List<CartLine>> มีเมธอดหลักอย่าง add(), setQty() และ setPrice() ทุกครั้งที่ข้อมูลเปลี่ยน เราเลือกสร้าง List ตัวใหม่ขึ้นมาแทนการแก้ไขของเดิม (immutable) แล้วค่อย emit() ออกไปให้ UI รับรู้ ส่วนกรณีที่ผิดเงื่อนไขทางธุรกิจ ฟังก์ชันจะคืนค่าเป็น String? เพื่อส่ง error กลับไปแสดงผลที่หน้าจอ

Clean Code Impact: จุดที่เห็นชัดที่สุดคือการยึด SRP (Single Responsibility Principle) ค่อนข้างเคร่งครัด ตรรกะการคำนวณและกฎทางธุรกิจถูกแยกออกจาก UI เกือบสมบูรณ์ คลาสนี้ไม่ผูกกับ Context หรือ Repository ภายนอกเลย ทำให้เขียน Unit Test ได้ง่ายขึ้นมาก และช่วยลด Coupling ของระบบโดยรวม



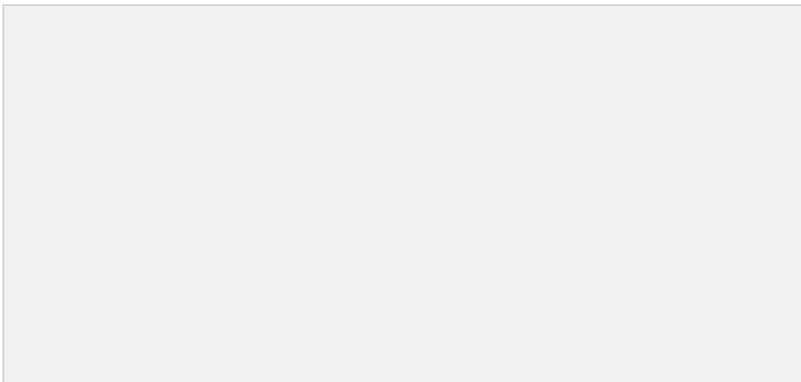
2. PendingQuoteCubit

(lib/presentation/blocs/pending_quote_cubit.dart)

ตัวนี้ทำหน้าที่เป็นเหมือนสะพานเชื่อมข้อมูล ระหว่างหน้าจัดการใบเสนอราคา (QuotesManager) กับหน้าตะกร้าสินค้า (Checkout) เวลาผู้ใช้กดเลือกใบเสนอราคาที่ต้องการแล้ว ข้อมูลจะถูกส่งผ่าน Cubit ตัวนี้แทนที่จะยัดผ่าน Route Arguments แบบเดิม

โครงสร้างเรียบง่ายมาก เก็บสถานะเป็น QuoteWithItems? ที่รองรับค่า Null ได้ มีแค่เมธอด set() สำหรับบรรจุข้อมูล และ clear() สำหรับล้างสถานะกลับเป็น Null หลังจากฝั่งตะกร้าดึงข้อมูลไปใช้เสร็จแล้ว

Clean Code Impact: ช่วยเรื่อง Separation of Concerns ในระดับสถาปัตยกรรมโดยตรง เพราะตัดปัญหาการส่งข้อมูลข้ามหน้าจอแบบยุ่งเหยิงออกไป แล้วย้ายมาตริงไว้ที่ State Management กลางแทน โค้ดทั้งไฟล์ไม่เกิน 25 บรรทัด และทำหน้าที่เดียวคือส่งผ่านข้อมูล ไม่มีอะไรเกินความจำเป็น



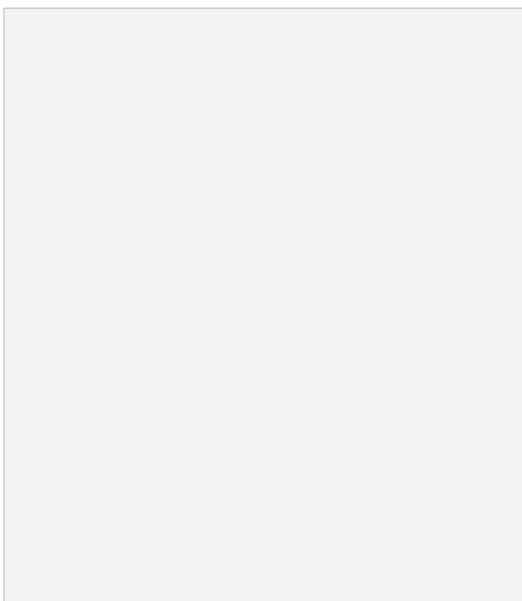
3. ThemeModeCubit

(lib/presentation/widgets/theme_controller.dart)

จัดการโหมดการแสดงผล Light/Dark ของทั้งแอป และดูแลเรื่องบันทึกค่าที่
ผู้ใช้เลือกไว้ลง Local Storage ด้วย เพื่อให้เปิดแอปครั้งต่อไปแล้วยังจำโหมดเดิมได้

เก็บสถานะเป็น ThemeMode ตอนเริ่มแอปจะมีการทำ Asynchronous I/O
เพื่อดึงค่าที่เคยบันทึกไว้ผ่าน SharedPreferences พอเรียก toggle() ระบบจะสลับ
สถานะปัจจุบันแล้ว emit() ปรับ UI ทันที จากนั้นค่อยเขียนข้อมูลกลับไปลง Storage
อยู่เบื้องหลังโดยไม่ทำให้ผู้ใช้ต้องรอ

Clean Code Impact: การผูก I/O เข้ากับ State Management ตรงนี้ช่วยตอบ
โจทย์ DRY ได้ดี เพราะไม่ว่าจะอยู่ตรงไหนของแอป ก็เรียกสลับ Theme ได้จากจุด
เดียวกัน ไม่ต้องเขียนโค้ดจัดการ SharedPreferences ซ้ำในหลายไฟล์เหมือนที่ AI
Gen เขียนมาแต่แรก



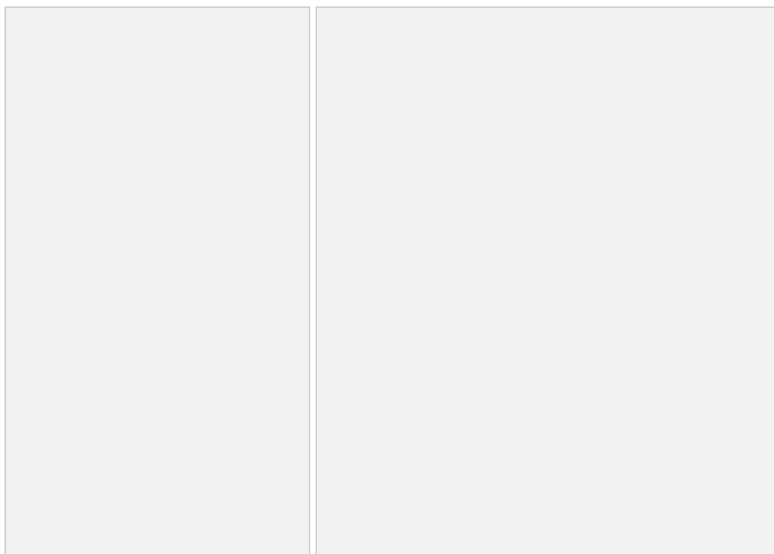
4. AppButton

(lib/presentation/widgets/app_button.dart)

เป็นปุ่มกดมาตรฐานของทั้งระบบ ครอบคลุมสไตล์ตาม Brand Identity ทั้ง Primary, Secondary และ Danger พร้อมรองรับสถานะ Loading ในตัวโดยไม่ต้องเขียนแยก

เป็น StatelessWidget รับพารามิเตอร์ AppButtonVariant เพื่อสลับ Render ระหว่าง FilledButton กับ OutlinedButton ถ้า busy == true ปุ่มจะถูก Disable และแสดง CircularProgressIndicator ทับตำแหน่ง Label แทน พร้อมสลับสีตัวอักษรให้เหมาะกับ Dark Mode ในบางประเภทของปุ่มด้วย

Clean Code Impact: สะท้อนความเป็น Modularity และหลัก DRY ได้ชัดเจน เพราะเป็น Single Source of Truth ของสี ระยะขอบ และ BorderRadius ทั้งระบบ ถ้าวันไหน Design System เปลี่ยน แก้ที่ไฟล์นี้ไฟล์เดียว UI ทั้งแอปก็อัปเดตตามทันที ไม่ต้องไล่แก้ทีละหน้า



5. EmptyState

(lib/presentation/widgets/empty_state.dart)

เป็น Widget Placeholder ที่เอาไว้แสดงตรงกลางหน้าจอเวลาไม่มีข้อมูล เช่น รายการสินค้าว่างเปล่า หรือค้นหาแล้วไม่เจอผลลัพธ์

จัดวางผ่าน Column ประกอบด้วย Icon, ข้อความหลัก (message), ข้อความรอง (hint) และพื้นที่สำหรับใส่ Widget ได้ตอบเพิ่มเติมถ้ามีการส่ง action เข้ามา มีการคำนวณระยะห่างและดึง Typography จาก Theme มาใช้ให้อัตโนมัติ

Clean Code Impact: ช่วยลด Boilerplate ที่มักจะเขียนซ้ำๆ เวลาสร้างหน้าจอเปล่า และเพิ่ม Readability ให้ฝั่ง UI ได้เยอะ เพราะใช้ Optional Parameters ทำให้โค้ดกระชับแต่ยังยืดหยุ่นพอสำหรับหลายสถานการณ์

